Instruction	Description	Example
`FROM`	Defines the base image for the subsequent instructions. It must be the first instruction in a Dockerfile.	`FROM ubuntu:22.04`
`LABEL`	Adds metadata to the image, following a key/value format. Multiple `LABEL` instructions can be defined.	`LABEL version="1.2.1"`
`ARG`	Defines a variable that users can pass at build time. Multiple `ARG` instructions can be defined.	`ARG JAR_FILE`
`RUN`	Executes the commands passed as arguments in a new layer on top of the existing ones. Multiple `RUN` instructions can be defined.	`RUN apt-get update && apt-get install -y default-jre`
`COPY`	Copies files or directories from the host filesystem to the one inside the container.	`COPY app-0.0.1-SNAPSHOT.jar app.jar`
`USER`	Defines the user that will run all the subsequent instructions and the image itself (as a container).	`USER sheldon`
`ENTRYPOINT`	Defines the program to execute when the image is run as a container. Only the last `ENTRYPOINT` instruction in a Dockerfile is considered.	`ENTRYPOINT ["/bin/bash"]`
`CMD`	Specifies defaults for an executing container. If the `ENTRYPOINT` instruction is defined, they are passed as arguments. If not, it should also contain an executable. Only the last `CMD` instruction in a Dockerfile is considered.	`CMD ["sleep", "10"]`

Once you have the specifications for creating a container image declared in a Dockerfile, you can use the `docker build` command to run all the instructions one by one, producing a new layer for each of them. The whole process from Dockerfile to image to container is illustrated in figure 6.4. Notice how the first instruction in a Dockerfile produces the lowest layer of the image.